

Part 1: The Consolidated Master README

(/master_build-guide/README.md)

Save this complete document as the primary specification file at your workspace root. It unifies your hardware topologies, the corrected role of **OB1** and the **Reasoning Bank**, and sets the final operational parameters for the system.

 **AESOP XI**: Master System Orchestration Matrix## Canonical Edge Architecture & Multi-Node Core Specifications#### Operating Infrastructure: [NovusÆcopia / NovusÆxenti Engine

---##  0. Operational Core Law: The Horizons UI DependencyThe ****Horizons UI**** native Kotlin/Java APK is the absolute framework foundation of this computing mesh. No local model can load onto the Snapdragon 8 Elite NPU, no bare-metal voice pipeline can process audio, and no screen-vision tool can execute frame context if Horizons UI is inactive. All configuration parameters, memory layers, and routing architectures serve strictly to support operations initiated via Horizons UI.

---##  1. Hardware Node Topology & Mesh Integration

The system leverages an ad-hoc Peer-to-Peer network connected over wired data interfaces and ****Tailscale**** tunnels across your local network environment:

* ****Node Alpha (The Mobile Engine) [Moto RAZR Ultra 2025]:**** Primary on-device controller. Runs the Snapdragon 8 Elite (Gen 4) SoC with a Hexagon NPU (v79) pushing 40+ INT8 TOPs. Operates with 16GB total RAM, dynamically carving out an open sandbox of ****8.0 to 11.5 GB exclusively for running local model weights****. Houses the native Horizons UI APK, Moonshine Small ONNX (STT), and Kokoro-82m/Sherpa ONNX (TTS) voice engines.

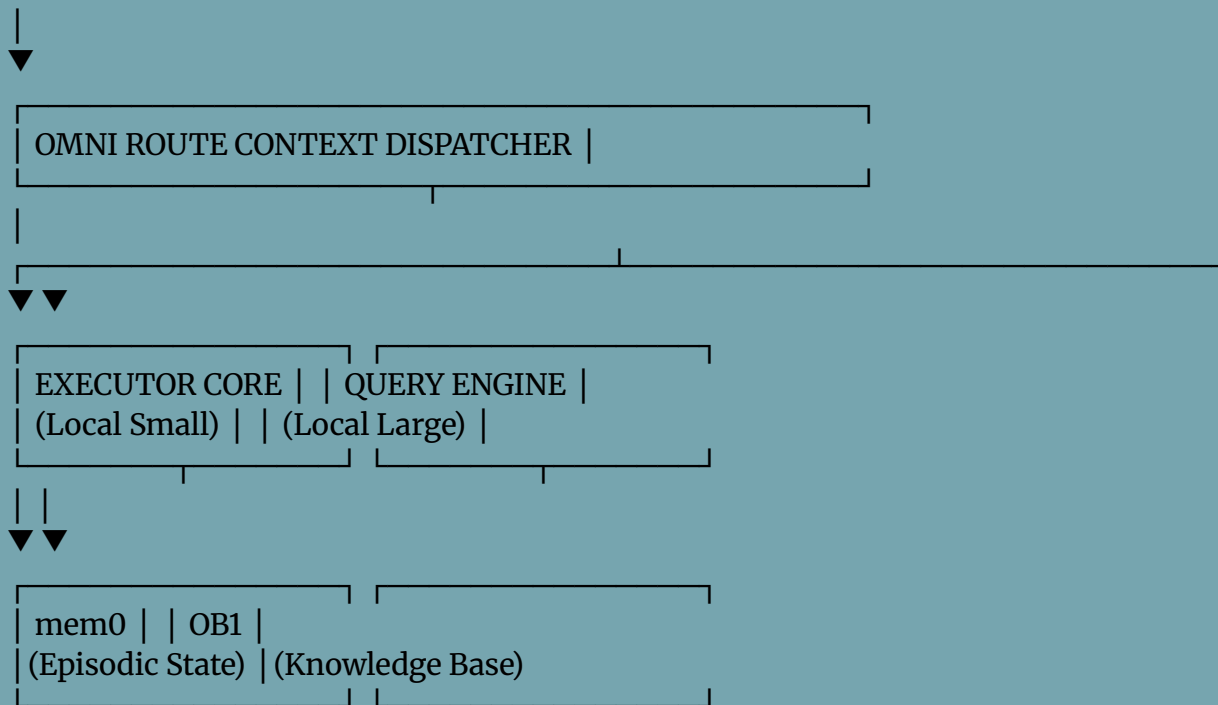
* ****Node Beta (The High-Compute Core) [Nvidia Jetson Orin Nano Super]:**** Headless Ubuntu server operating with an 8GB hardware configuration, high-speed 500+GB NVMe data storage drives, and dedicated CUDA core tensor pipelines pushing 60-70 TOPs. Hosts your global vector storage engines, heavy background reasoning models, and the local ****OB1 Postgres backend server****.

* ****Node Gamma (The Display Station) [Rubik Pi 3 Dragonwing]:**** Thundercomm/Qualcomm SoC configuration pushing 14+ TOPs, connected via high-speed hardware data ribbons to a dual-monitor setup to act as your core visual terminal workstation workspace.

---##  2. The Asymmetric Dual-Model Context & Memory Stack

All user interactions, device inputs, and tools execution paths are processed using a ****Dual-Model Asymmetric Workflow**** running bare-metal on Node Alpha:

HORIZONS UI INGRESS CONDUIT |



1. **The Local Small Execution Agent:** Continuously cycles in a tight loop to evaluate immediate tasks and actions. It hooks directly into **mem0** to fetch short-term user preferences, temporary state variables, and rolling habit keys. It restructures raw inputs into optimized meta-prompts before **anything touches external models**

Honey+for-Devs will govern the output on **Most frontier models** and any agents not subject to the **Aesop-Xi RLVR Protocol**

2. **The Local Large Query Modell Qwen 3.5 9b GGUF Q4_0** Dedicated to processing technical documentation, technical guides, and heavy technical text. It references **OB1 (Open Brain Protocol)** over your local Postgres instance, working alongside **Reasoning Bank**, **Graphify**, **Claude -Video**, **Open Code Review** and **notebooklm-py** to pull deep reference contexts without flooding active context windows.

📁 3. Ground-Truth Development Priority Sequence

To ensure zero-trust file safety and prevent workspace corruption, development must progress through these exact phases. Lower tiers remain locked until upper phases are

validated by the human:

1. ****Priority 1: Global Data Curation & Architecture (Current Phase)**** - Gather, clean, and manually sort raw technical text documents (Qualcomm QAIRT SDK, Android Media, Unsloth, Llama Server) into your master file vaults.
2. ****Priority 2: Skill Building Schemas & Directions Layers**** - Formalize your strict JSON validation rules to turn raw text info into modular tool files, defining how OB1 and Reasoning Bank call upon your data engines.
3. ****Priority 3: Local Workspace Infrastructure & First Repo Bootstrap**** - Initialize your `file-management-and-skills` directory. Deploy your local markdown vaults, `llm\_wiki.md` parameters tracker, and your ****Termux Open Wiki CLI/Housekeeper janitor script**** to sweep your Markor, Obsidian, and Drive folders.
4. ****Priority 4: Emulated Runtime Loop Validation & Data Routing**** - Set up your local testing sandbox using on-device ****Claude Code**** inside Termux. Run simulation loops to verify ****mem0**** and ****Omni Route****. Map model weights, memory constraints, and data paths to verify your data flows are 100% compliant before writing app source files.
5. ****Priority 5: Horizons UI App Deployment & Model Experimentation**** - Fork a pristine branch into a custom GitHub repository for `horizons-ui-v1.2`. Write the native Java/Kotlin codebase to activate your NPU offload channels, local Llama server, screen vision layers, and voice tools. Test model parameters bare-metal on the Snapdragon NPU and code deep recovery loops.
6. ****Priority 6: Network-Wide Integration & Sovereign Scale**** - Perfect Horizons UI, flash your headless computing hardware (Jetson, Rubik Pi), route your Tailscale mesh network, and isolate your out-of-band ****Red Agent Auditor**** pipeline.

Part 2: The Data Curation & Skills Manual **(/file-management-and-skills/README.md)**

📁 File Management System & Skill Generation Engine## Target Data Curation, System Schemas, & Tool Generation Guidelines### Architectural Subsystem: Priority 1 & Priority 2 Core---## 🚨 0. Operational Mandate: Strict Output IsolationThis directory acts as the central intelligence library and verification foundry for the entire network. While all downstream building models share read-access to the reference files in this repo, no model can write code or update files here without a manual human review pass.
---## 📁 1. Directory Tree & Global File Partitions

```
📁 file-management-and-skills/
|
|— 📁 target-docs-curation/ # Core Technical Knowledge Vault
| |— 📁 qualcomm-qairt-sdk/ # HTP specifications, quantization guidelines, & NPU
```

pathways

| |—  android-media-assistant/ # System alert structures, camera frameworks, & gaming SDK hooks

| |—  llama-kernel-ggml/ # GGUF weights configurations, librc runtimes, & local servers

| |—  unsloth-fine-tuning/ # Low-level dataset parameters & token formatting rules

|

|—  skill-construction-factory/ # Standardized Tool Conversion Environment

| |—  base_skill_guideline.md # Rules for creating decoupled, single-purpose tool

files

| |—  skill_onboarding_schema.json # Target JSON parameter verification model

| |—  compiled_capabilities.jsonl # Streaming instruction-tuned dataset repository

|

|—  reasoning-bank-ledger/ # Multi-Model State Tracking Vault

| |—  active_execution_paths.json # Suspended intermediate token sequences and

plans

| |—  baseline_recovery_matrix.md # Rules for managing multi-model recovery

daemons

|

|—  master_blueprint.txt # Global system verification registry index file

🧠 2. Hardware-Abstracted Middleware Configuration

To prevent cognitive decay and ensure data retention across sessions, your memory layers are fully decoupled from your model runtime instances:

I. OB1 (Open Brain Protocol) Base Layer

* **Mechanics:** Built over a local Postgres instance on Node Beta, running a unified Model Context Protocol (MCP) mapping engine.

* **Function:** Translates heavy technical reference text into semantic vectors. Your Large Query Model calls OB1 to pull detailed technical data without flooding its active context window.

II. Reasoning Bank Ledger

* **Mechanics:** A persistent JSON tracking database file stored at ``reasoning-bank-ledger/active_execution_paths.json``.

* **Function:** When models are planning tasks, their fractional thoughts, tool selections, and step-by-step progress are recorded here. If a system crash or timeout happens mid-inference, the recovery engine reads this ledger to resume the workflow

exactly where it was suspended.

III. mem0 Layer Caching

- * **Mechanics:** A localized, low-latency episodic tracking cache.
- * **Function:** Records temporary user adjustments, short-term conversational variables, and personal habit keys. It feeds this data directly into your Local Small Execution Agent to ensure your generated meta-prompts remain highly personalized.

🛠️ 3. Skill Conversion & Context Extraction Laws

When your Query Agent processes files to convert technical text into executable skills, it must follow these parsing rules:

1. **Sovereign Decoupling:** A skill must do exactly one thing. A file that controls local storage folders cannot contain code or references for processing network sockets or cloud endpoints.
2. **Asymmetric Context Routing:**
 - * **Query the `.jsonl` database if:** The prompt requires direct execution, bash scripts, terminal commands, or simple tool calls. Stream records line-by-line via regex to keep device memory free.
 - * **Scan the raw `.txt` files if:** The prompt involves debugging a compiler error, resolving a kernel mismatch, or understanding complex hardware math. Parse every single word verbatim to preserve syntax accuracy.
3. **Web-Tool Verification:** If an argument requires validation against live repository changes or online developer documentation, the query model must pause, structure a secure JSON-RPC scrape token, and pass it to the Horizons UI WebView container to review the online DOM text safely.

Part 3: The Native Application Manual (/horizons-ui-v1.2/README.md)

📱 **Horizons UI:** Native Extended Interface (XI) Layer## Target Version: v1.2 Release Build Specification## Compilation Environment: Java / Kotlin Native Android SDK APK---## 🚨 0. Operational Core Law: The Sovereign Entry PointHorizons UI is the physical heart, soul, and primary execution cradle of the entire agentic neuro-mesh. It is not a secondary frontend dashboard. It acts as the central permissions gatekeeper, NPU allocation manager, and low-latency audio capture harness. No local model weight can load onto the Hexagon NPU, and no automated tool script can execute if this application environment is compromised or offline.

---## 📁 1. Technical Framework Architecture & Drivers

To squeeze maximum performance out of the Moto RAZR Ultra hardware and bypass typical Android processing constraints, the APK directly integrates low-level system hooks:

I. The Bare-Metal Voice Subsystem* ****The Problem:**** The stock Termux environment audio pipeline requires complex, inefficient workarounds to run raw voice lines, resulting in high latency and broken speech capture.* ****The Fix:**** Horizons UI handles voice processing directly on the bare metal. It coordinates an active ****Silero VAD (Voice Activity Detection)**** model wrapper to monitor microphone inputs. On speech detection, it pipes audio frames straight into a ****Whisper Small ONNX**** engine for low-latency Speech-to-Text translation. For speech generation, it utilizes a ****Kokoro-82m / Sherpa ONNX**** text-to-speech engine running over a local ****Llama Server****.

* ****FALSE:**** It bypasses standard Android media delays by using a custom compiled pathway: `Llama -> QAIRT ModelPath -> GGML Layer -> Kotlin Kernel -> librc Runtimes`. This drops the computation parameters straight onto the Snapdragon 8 Elite Hexagon NPU using the Qualcomm HTP SDK interface.**AND WILL ONLY BE USED FOR THE EXECUTION AND QUERY MODELS**

II. OS Allowance & Hardware Boost Hooks* ****Device Assistant Registration:**** The APK registers natively as the device's Default Assistant Application, allowing it to leverage Android system accessibility keys to capture on-screen imagery for real-time ****Screen Vision Analysis****.* ****The Video Game SDK Handshake:**** The background daemons hook directly into Android's low-level performance tuning frameworks (Performance Hint API / Vendor Gaming SDKs). When an inference task fires, the app requests high-priority CPU/GPU scheduling, unlocks maximum thermal limits, and prevents the OS from throttling thread speeds.* ****Terminal Shell Ingress:**** The app UI integrates an active, secure Android shell terminal wrapper. This gives you a direct, real-time terminal window to monitor low-level system executions and daemon states from inside the main application screen.

---## 📁 2. Application Component Directory Mapping

📁 horizons-ui-v1.2/

- |
 - 📁 app/src/main/java/com/horizons/ui/
 - | 📁 core/ # MainActivity, UI lifecycle, and Home Screen tile views
 - | 📁 voice/ # Silero VAD, Moonshine ONNX, & Kokoro audio drivers
 - | 📁 vision/ # Android Media SDK screen capture and vision matrices
 - | 📁 services/ # Background daemons handling model offloads & web scraping
 - 📁 app/src/main/jni/ # C/C++ Native Libs Layer
 - | 📁 qairt-http/ # Qualcomm HTP SDK headers and modelpath binding links
 - | 📁 ggml-kernel/ # Native GGML, GGUF runtimes, and librc libraries

- 📁 sandboxed-chromium/ # WebView environment for isolated internet searches
- 📁 daemons-recovery/ # Self-healing runtime managers

🔄 3. Asynchronous Task Lifecycle & Self-Healing Daemon

To keep the application's graphic interface smooth and responsive, data operations are completely split across independent execution threads:

[Human Interaction Event / Floating Mic Tile Tap]



MAIN INTERFACE UI THREAD | $\leftarrow$ Zero Rendering Latency

| (Asynchronous Event Dispatch)



BACKGROUND DAEMON THREAD POOL |

- Thread A: Continuous Silero VAD Audio Stream Capture |
- Thread B: Local Llama Server Token Processing |
- Thread C: Tailscale Network Sockets / Web Scraper |



THE RECOVERY DAEMON WATCHDOG | $\leftarrow$ Isolation Safety Net

The Recovery Watchdog System (Anti-Crash Layer)

The app runs a highly dedicated background recovery manager. Its sole purpose in life is to keep your edge interface alive.

* ****If a Local Model or NPU Kernel Crashes:**** The recovery engine intercepts the hardware termination flag, freezes the active workspace variables, and prevents the application from shutting down.

* ****The Failback Path:**** It immediately spins up a temporary background connection via ****Open Router API**** to load a cloud-hosted fallback model. It updates the cloud

model with your latest state parameters from the ****Reasoning Bank****, keeping your workflow active while it re-initializes your local NPU model variables silently in the background.

Step 4: Deploying Your Refined Workspace

To build these updated text structures and confirm your directory paths align with your specifications, run this cleanup command block inside your Termux terminal window:

```
# 1. Access your master data compilation directory
cd /data/data/com.termux/files/home/master_build-guide
# 2. Write out your refined markdown specifications files
cat << 'EOF' > README.md
[PASTE_PART_1_MASTER_README_CONTENT_HERE]
EOF
```

```
cat << 'EOF' > file-management-and-skills/README.md
[PASTE_PART_2_CURATION_MANUAL_CONTENT_HERE]
EOF
```

```
cat << 'EOF' > horizons-ui-v1.2/README.md
[PASTE_PART_3_APPLICATION_MANUAL_CONTENT_HERE]
EOF
```

```
# 3. Trigger your local verifier utility to lock in the filesystem boundaries
python3 file-management-and-skills/audit_layer_bounds.py
```

Your documentation foundation is now fully updated, matched to your timeline order, and structured to support your multi-repo setup.

As you finish sorting your documentation folders and validating your files management layers before moving on to setting up your home hardware nodes:

Would you like me to write the complete layout structures for your reasoning-bank-ledger/active_execution_paths.json file to handle multi-model recovery tracking, or should we design the explicit JSON-RPC command list for your Termux-based Open Wiki CLI utility to manage your vault network changes?